

Anti-patrones en Go

Por: Ricardo Cuellar



God struct

```
example.go

// God struct – un struct que hace demasiado
type App struct {
    DB          *pgxpool.Pool
    Redis       *redis.Client
    Logger      *slog.Logger
    JWTSecret   string
    S3Client    *s3.Client
    EmailClient *email.Client
    Validator   *validator.Validate
    // ... y mil más más
}

func (a *App) CreateUser(email, password string) error { /* ... */ }
func (a *App) SendEmail(to, subject, body string) error { /* ... */ }
func (a *App) UploadFile(name string, data []byte) error { /* ... */ }
func (a *App) GenerateReport() ([]byte, error) { /* ... */ }
// ... 40 métodos más, todos colgando del mismo struct
```



God struct - Corregido

```
example.go

// Responsabilidades separadas, cada una con su propia interfaz
type UserService struct {
    repo UserRepository
}

type EmailService struct {
    client EmailClient
}

type FileService struct {
    storage Storage
}

// main.go arma las piezas, pero ningún struct individual
// conoce o depende de los demás
```



Dependencias circulares



example.go

```
// Esto NO compila en Go – dependencia circular  
// user/user.go  
package user  
import "devboard/internal/task" // mal D:  
  
// task/task.go  
package task  
import "devboard/internal/user" // mal ciclo!
```



Dependencias circulares - corregido

```
example.go

// Solución: extraer lo compartido a un package común,
// o invertir la dependencia con una interfaz
// domain/task.go – no depende de nada externo
package domain

type Task struct {
    ID          string
    AssigneeID  string // referencia por ID
}

// El package task puede usar domain.Task sin necesitar
// importar el package user directamente
```



{dev/talles}



init()

```
example.go

// init() que hace cosas no obvias
package config

var DatabaseURL string

func init() {
    DatabaseURL = os.Getenv("DATABASE_URL")
    if DatabaseURL == "" {
        log.Fatalf("DATABASE_URL no configurada")
    }
}
```



init() - corregido

```
example.go

//Validación explícita, llamada conscientemente desde main()
package config

type Config struct {
    DatabaseURL string
}

func Load() (*Config, error) {
    dbURL := os.Getenv("DATABASE_URL")
    if dbURL == "" {
        return nil, errors.New("DATABASE_URL no configurada")
    }
    return &Config{DatabaseURL: dbURL}, nil
}
```

```
example.go

// En main.go - explícito y visible
func main() {
    cfg, err := config.Load()
    if err != nil {
        log.Fatalf("error cargando configuración: %v", err)
    }
    // ...
}
```



Errores sin wrap

```
example.go

// Perder el contexto del error original
func (r *TaskRepository) GetByID(id string) (*Task, error) {
    var task Task
    err := r.db.QueryRow(query, id).Scan(&task.ID, &task.Title)
    if err != nil {
        return nil, errors.New("error obteniendo tarea")
        // El error original (¿fue ErrNoRows? ¿timeout? ¿conexión perdida?)
    }
    return &task, nil
}
```



Errores sin wrap - corregido

```
example.go

// Preservar el error original con wrap
func (r *TaskRepository) GetByID(id string) (*Task, error) {
    var task Task
    err := r.db.QueryRow(query, id).Scan(&task.ID, &task.Title)
    if err != nil {
        return nil, fmt.Errorf("obteniendo tarea %s: %w", id, err)
        // Ahora el log muestra el error real,
        // Y errors.Is sigue funcionando en capas superiores
    }
    return &task, nil
}
```



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

